Chunking

What is Chunking?

Chunking is the process of dividing large data into smaller, meaningful units called chunks.

In a RAG pipeline, each chunk can be:

- Embedded separately
- Stored separately
- Retrieved separately
- Sent to an LLM only when relevant

Basic flow

Large document

↓

Chunking

↓

Chunk 1, Chunk 2, Chunk 3...

↓

Embeddings and retrieval

↓

Relevant chunks sent to the LLM

Simple example

Suppose the original document contains three topics:

Section 1: What is artificial intelligence?

Section 2: What is machine learning?

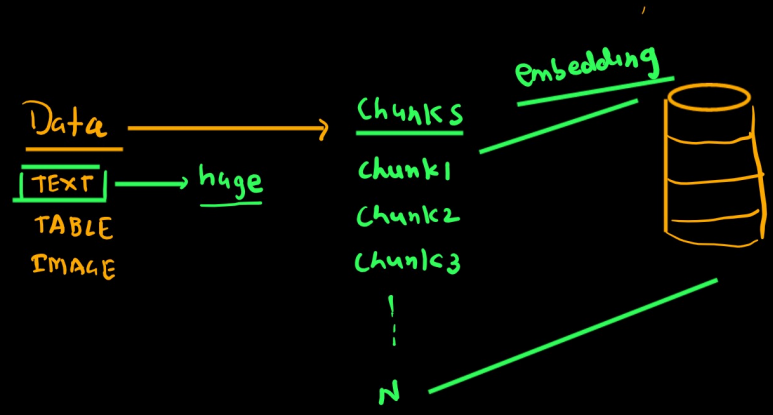
Section 3: What is deep learning?

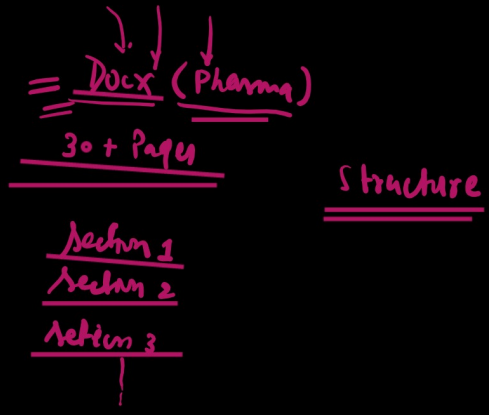
Instead of storing the complete document as one unit, we can create:

Chunk 1 → Artificial intelligence section

Chunk 2 → Machine learning section

Chunk 3 → Deep learning section





Why Do We Need Chunking?

2.1 LLM context-window limitations

Every LLM has a maximum input capacity. A very large document may not fit inside one request.

2.2 Better retrieval accuracy

A small relevant section is easier to retrieve than one huge document containing many unrelated topics.

2.3 Lower token usage and cost

Only relevant chunks need to be sent to the LLM.

2.4 Faster processing

Smaller units are easier to embed, index, retrieve, and process.

2.5 Sending only relevant information

The LLM receives focused context instead of the entire knowledge base.

Important balance

- Very small chunks may lose context.
- Very large chunks may contain irrelevant information.
- Excessive overlap creates duplication and increases storage and token usage.

Is Chunking Always Required?

No. Chunking is a design choice, not a compulsory step.

Chunking is usually required when:

- The document is large
- The document exceeds the model context window
- Semantic retrieval is required
- Different sections discuss different topics
- Only a small section should be sent to the LLM
- The source is a long PDF, manual, policy, or knowledge base

Chunking may not be required when:

- The input is already short
- One complete record is already a meaningful unit
- SQL can retrieve exact rows
- The full document is required for the task
- The complete input comfortably fits inside the model context
- The data is already separated into FAQs, tickets, products, or database records

LangChain splitter classes

1. RecursiveCharacterTextSplitter
2. TokenTextSplitter
3. CharacterTextSplitter
4. MarkdownHeaderTextSplitter
5. HTMLHeaderTextSplitter
6. HTMLSemanticPreservingSplitter
7. NLTKTextSplitter
8. SpacyTextSplitter
9. RecursiveJsonSplitter
10. PythonCodeTextSplitter

Fixed-size Chunking
Content-aware Chunking
Document Structure-Based Chunking
Semantic Chunking

section | layout | semantic

Chunk Size and Chunk Overlap

300 char 300 token hyperparameter

Chunk size

The maximum target amount of content in one chunk.

Chunk overlap

The amount of repeated content between neighboring chunks.

Overlap helps when an important sentence or concept falls near a chunk boundary, but excessive overlap increases duplication.

- Smaller chunk size usually creates more chunks.
- Larger overlap usually increases repeated stored content.
- A larger chunk may preserve more context but reduce retrieval precision.
- A smaller chunk may improve precision but lose surrounding explanation.
- The best values must be evaluated using real questions and retrieval results.

Suppose:

Chunk size = 10 words
Chunk overlap = 3 words

hyperparameter → we can not decide the value

experimental

Original text:

Artificial intelligence helps machines learn patterns and make useful decisions from data.

The chunks may look like:

Chunk 1:
Artificial intelligence helps machines learn patterns and make useful decisions

Chunk 2:
make useful decisions from data

Here, the words:

make useful decisions

are repeated in both chunks. This repeated part is called chunk overlap.

It helps preserve the meaning when an important sentence is divided between two chunks.

Common mistake while chunking

Choosing chunk size without evaluating retrieval

Creating chunks that are too small

Creating chunks that are too large

Adding excessive overlap

Splitting headings away from their content

Splitting tables without preserving headers

Splitting code in the middle of functions or classes

Removing useful metadata

Using semantic or agentic chunking without comparing a simpler baseline

Assuming one strategy works for every data type

Chunking Beyond Text

Chunking is not limited to text. We can also divide images, tables, code, audio, video, maps, medical scans, and document layouts into smaller meaningful units.

The chunking boundary depends on the type of data.

Image Chunking

Image chunking means dividing a large image into smaller visual regions.

An image can be divided using:

- Fixed tiles
- Overlapping tiles
- Detected objects
- Document-layout regions
- Semantic regions

Pattern

For example, a scanned document image may be divided into:

- Header
- Paragraph
- Table
- Chart
- Image
- Footer

Why do we chunk images?

Image chunking is useful when:

- The image is very large
- Small text may disappear after resizing
- Tiny objects need to be detected
- The image contains many independent regions
- Region-level retrieval is required
- The image is a map, medical scan, document, diagram, or satellite image

Example

Suppose a large document image contains a paragraph, table, and chart.

Instead of processing the entire image as one unit, we can create:

Chunk 1 → Paragraph region
Chunk 2 → Table region
Chunk 3 → Chart region

Now each region can be processed separately.

Important limitation

Image chunking may lose the overall relationship between different regions.

Therefore, a strong system may store both:

Full image → Global context

Image chunks → Local details

Table Chunking

Table chunking means dividing a large table into smaller logical table sections.

A table can be chunked by:

Rows
Overlapping rows
Columns
Categories
Entities
Dates or time ranges
Row-based chunking

A large table can be divided into groups of rows.

Example:

Chunk 1 → Rows 1 to 3
Chunk 2 → Rows 3 to 5
Chunk 3 → Rows 5 to 7

The repeated rows represent overlap.

This is useful when nearby records are related, such as transaction history, logs, sensor readings, or patient records.

Category-based chunking

Rows can be grouped according to their category.

Example:

Chunk 1 → Electronics products
Chunk 2 → Furniture products
Chunk 3 → Stationery products

This is often more meaningful than splitting the table after an arbitrary number of rows.

Column-based chunking

A very wide table can be divided into groups of related columns.

Example:

Chunk 1 → Product information
Chunk 2 → Sales information
Chunk 3 → Location information

A common key such as Order_ID should be preserved in every chunk so that the chunks can still be connected.

What must be preserved in every table chunk?

Every table chunk should preserve:

Column names
Primary or joining keys
Table title
Units
Relevant metadata
Row and column meaning

Without column names, values may become unclear.

For example:

O-101, Laptop, 2, 2400

is less meaningful than:

Order ID: O-101
Product: Laptop
Quantity: 2
Revenue: $2,400
How table chunks are used in RAG

Embedding models generally work better with descriptive text.

Therefore, a table chunk is often converted into text such as:

Table: Sales Orders

Order ID: O-101
Product: Laptop
Category: Electronics
Region: North
Quantity: 2
Revenue: $2,400

This text can then be embedded, stored, and retrieved.

When should we not embed a table?

When the data is already stored in a SQL database and the user wants exact calculations, filters, joins, totals, or averages, SQL is usually the better solution.

User question
↓
SQL query
↓
Exact rows or calculated result
↓
LLM explanation

For example:

What is the total revenue?
Which product generated the highest sales?
Show all orders from the North region.

These questions are better handled using SQL.

Vector retrieval is more useful when:

The table comes from a PDF
The table is not stored in a database
Semantic matching is required
The user asks meaning-based questions
Exact SQL querying is unavailable

Chunking can be applied to text, images, tables, code, audio, and video.

Image chunking divides a large image into smaller visual regions so that small text, objects, tables, and charts can be processed separately.

Table chunking divides a large table by rows, columns, categories, entities, or dates. Every table chunk must preserve headers, keys, units, and metadata.

However, chunking is not always the best option. For exact structured-data questions, SQL is usually better than vector retrieval.

recursive_splitter = RecursiveCharacterTextSplitter(separators=["\n\n", "\n", ". ", " ", ""], chunk_size=300, chunk_overlap=50, length_function=len, add_start_index=True,)

separators=["\n\n", "\n", ". ", " ", ""]

RecursiveCharacterTextSplitter splits text recursively by using multiple separators in priority order.

It first tries:

"\n\n"

This means it tries to split the text at paragraph boundaries.

If a paragraph is larger than 300 characters, it tries:

"\n"

This means it tries to split the paragraph at line boundaries.

If a line is still too large, it uses:

". "

This means it tries to split the text at sentence boundaries.

If a sentence is still too large, it uses:

" "

This means it splits the sentence between words.

The final fallback is:

""

This means it can split the text at the individual-character level when necessary.

chunk_size=300

means that the final chunks should be approximately no more than 300 characters.

chunk_overlap=50

means that approximately 50 characters from the previous chunk are repeated in the next chunk so that context is not lost.

length_function=len

means that chunk size is calculated using the number of characters.

add_start_index=True

stores the character position where each chunk starts in the original document as metadata.

This approach tries to preserve paragraph, line, sentence, and word boundaries, which makes it useful for general RAG applications.

TokenTextSplitter divides text according to tokens, not characters, words, paragraphs, or sentences.
encoding_name="cl100k_base" selects the tokenizer used to convert text into tokens.
chunk_size=10 means every chunk can contain a maximum of 10 tokens.
chunk_overlap=2 means the last 2 tokens of one chunk are repeated in the next chunk.

text: AI helps machines learn from data and make better decision for users
token: [AI] [helps] [machines] [learn] [from] [data] [and] [make] [better] [decisions] [for] [users]

chunk_size = 5 tokens
chunk_overlap = 2 tokens

Chunk 1:
AI helps machines learn from

Chunk 2:
learn from data and make

Chunk 3:
and make better decisions for

Chunk 4:
decisions for users

Suppose the document contains:

Artificial Intelligence

AI enables machines to perform intelligent tasks.
Machine learning is a part of AI.

Employee Leave Policy

Employees receive 20 paid leave days.
Leave must be approved by the manager.

A fixed-size splitter may create:

Chunk 1:
AI enables machines to perform intelligent tasks.
Machine learning is a part of AI. Employee Leave

Chunk 2:
Policy. Employees receive 20 paid leave days...

Content-aware chunking creates:

Chunk 1:
Artificial Intelligence

AI enables machines to perform intelligent tasks.
Machine learning is a part of AI.

Chunk 2:
Employee Leave Policy

Employees receive 20 paid leave days.
Leave must be approved by the manager.

How Content-Aware Chunking Works

Content-aware chunking splits text by understanding its natural structure, instead of cutting it blindly after a fixed number of characters or tokens.

It tries to preserve boundaries such as:

Paragraphs
Sentences
Headings
Lists
Sections
Tables
Code blocks

semantic_chunking()
↓
create_sentence_windows()
↓
create_local_embeddings(windows)
↓
SentenceTransformer(model_name)
↓
model.encode(windows)
↓
embeddings
↓
similarity and distance calculation
↓
topic-change boundaries
↓
semantic chunks